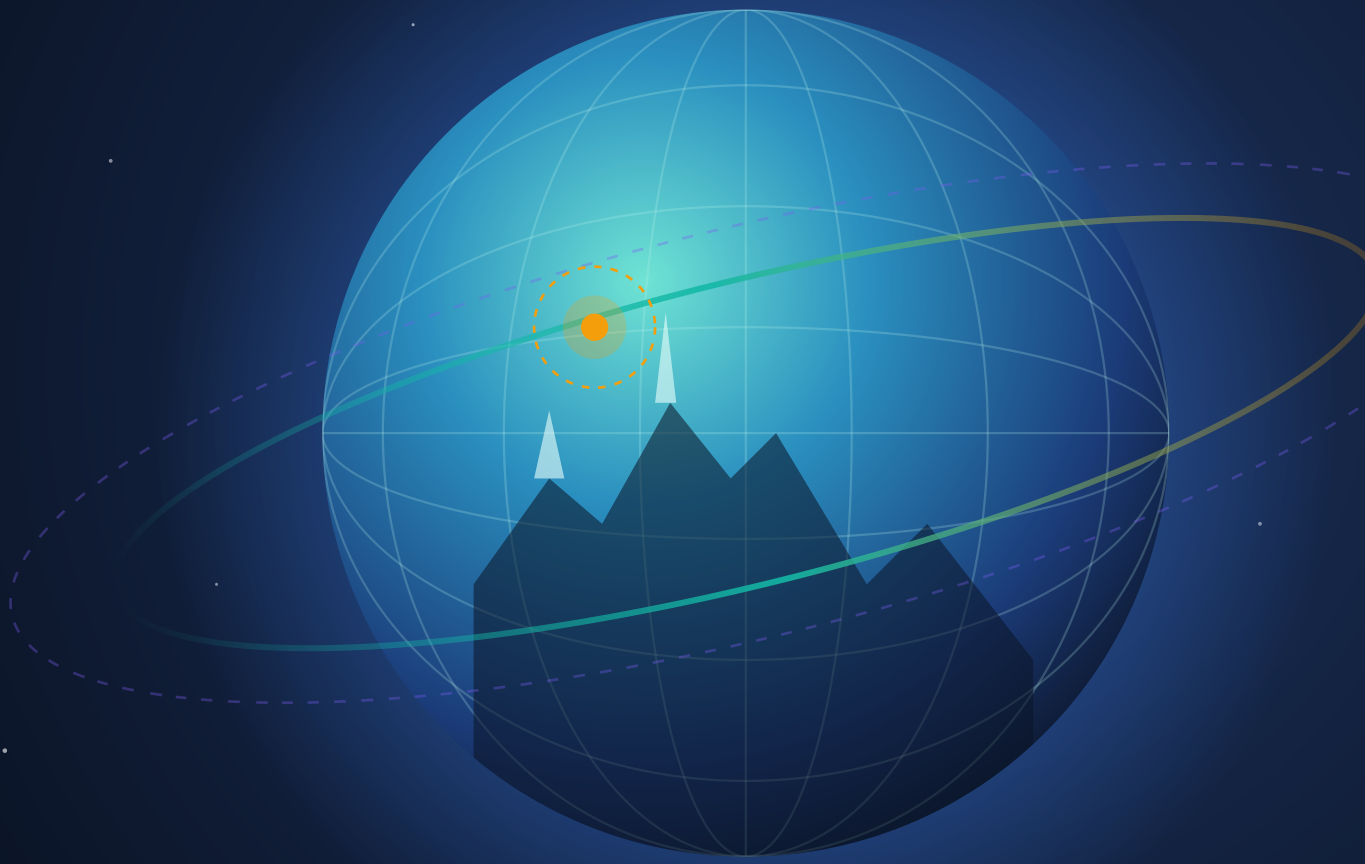


TECHNICAL DOCUMENTATION

Planet Render 360

Render any point on the map as a geographically faithful 3D, 360-degree view — built on device from free public data, in Godot.

ENGINE	TARGETS	EDITION	LICENSE
Godot 4.7 · GDScript	**macOS · iOS · Android**	**October 2026**	**MIT**

CONTENTS

What is inside

This document describes how Planet Render 360 turns a latitude and longitude into a walkable-looking panorama: the architecture, the data it fetches, the staged generation pipeline, the world model, rendering, caching and verification.

01	Overview & principles	Goals, hard constraints, layered architecture	3
02	Screens & user flow	Screen manager, state transitions, UI	4
03	Data sources	Nine free public sources, register gate	5
04	Generation pipeline	Five stages, overlapped fetches, fallbacks	6
05	World model	LocationData, WorldScale, GroundTruth	7
06	World assembly	Builder order, buildings, vegetation, landmarks	8
07	LOD & rendering	Static LOD rings, atmosphere, sun	9
08	Quality presets & governor	Fast → Ultra, FPS-driven stepping	10
09	Caching & offline	Data cache, built-world cache, history	11
10	Landmarks	Curated registry and OSM archetypes	12
11	Code organisation & testing	Packages, size budget, test suite	13
12	Gallery & attribution	Captures, licences, credits	14

At a glance

96 GDSCRIPT CLASSES	**23.2k** LINES OF CODE	**31** HEADLESS TESTS	**9** FREE DATA SOURCES
5 PIPELINE STAGES	**2.4 km** DETAILED EXTENT	**33 km** DEM HORIZON	**10** BESPOKE LANDMARKS

> Source of truth: this PDF summarises the code and the living documents in `docs/` (world-model contract, data-source register, fidelity scorecard) and `.agents/plans/`. Where they differ, the code and the contract win.

01 · OVERVIEW

Overview & principles

Pick a place by tapping a world map, searching, or typing coordinates. The app fetches real elevation, imagery, map features and climate for that point, builds a 3D world in true metres around it, and drops you in the middle with a drag-to-look 360° camera.

Hard constraints

- **No self-hosted server.** The device talks directly to free public sources; no proxy, account or backend.
- **Free, no paywall.** No purchases, subscriptions or locked tiers; no paid API keys.
- **Mobile-first.** Smooth on a mid-range phone; desktop may be prettier, mobile budgets win.
- **Few-minute loading.** Progress is real per-stage work, never a fake timer.

Design invariants

- **One unit is one metre** — horizontally and vertically, end to end.
- **Real data first.** Procedural rules run only on *absence* of real data for a layer, never on top of it.
- **Data contracts before visuals.** Generation never parses provider JSON or PNG; fetchers normalise into `LocationData`.
- **Attribution is UI.** Credits shown in-app come from the fetchers actually used.

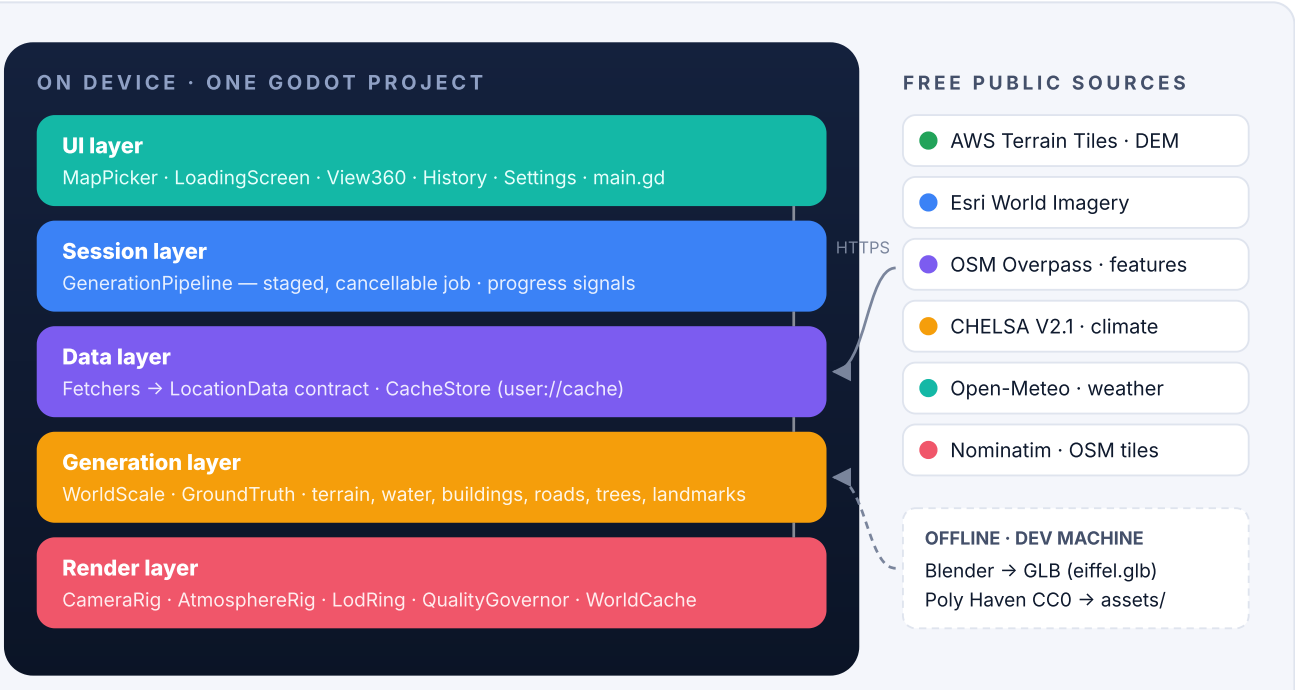


Figure 1. Layered architecture. Every layer lives in one Godot project; the only network traffic goes straight from the device to the public sources listed in the data-source register.

The stack is deliberately small: Godot owns UI, networking and rendering (no Flutter, no web view); GDScript is the only runtime language; Blender and GDAL are developer-machine tools whose output is committed to `assets/`. Godot's renderer targets Vulkan/Metal-class devices with Compatibility as the fallback.

Screens & user flow

`scripts/main.gd` is the screen manager. It owns the pipeline, the cache store and the settings, wires their signals to the screens, and shows exactly one screen at a time.

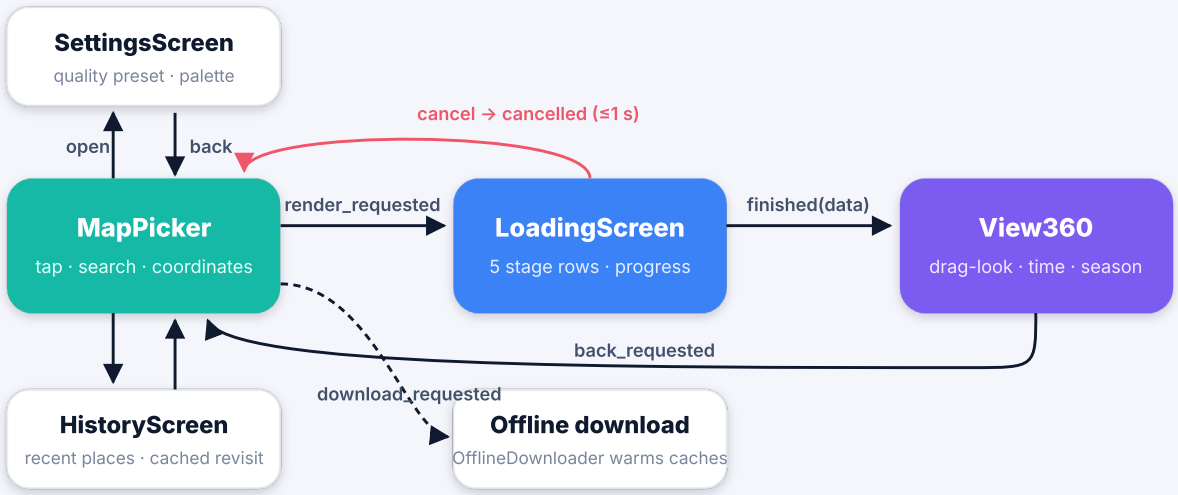
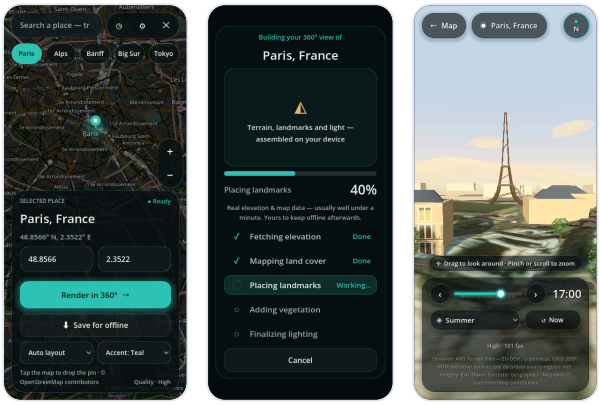


Figure 2. Screen state machine. A generation id is bumped on every start and cancel, so a world still building on its thread is dropped once superseded; starts are ignored while a run is in flight.

Interaction model

- **Map picker** — a Godot-native map of real OSM raster tiles (`MapTileView`), centred on the selection; a progressive preview loads first, then detail centre-out.
- **Search** — Nominatim, user-triggered only (no autocomplete), one request in flight, plus a bundled gazetteer and raw coordinates.
- **Loading** — five stage rows driven by `stage_changed` and `progress_changed`; cancel returns to the picker within a second.
- **360 view** — drag to look (yaw wraps, pitch clamps), pinch/scroll zoom, time-of-day slider with a "now" button, season and weather conditions, compass and attribution.
- **Responsive** — one 390×844 pt design frame, `canvas_items` stretch, and phone / tablet / desktop breakpoints in `responsive.gd`.



Phone frame: picker, loading, 360 view.

03 · DATA

Data sources

No source enters the fetch layer until its row in `docs/data-source-register.md` is complete: endpoint, format, licence, direct-mobile access, free-without-key, and a fallback. Each source has exactly one fetcher module.

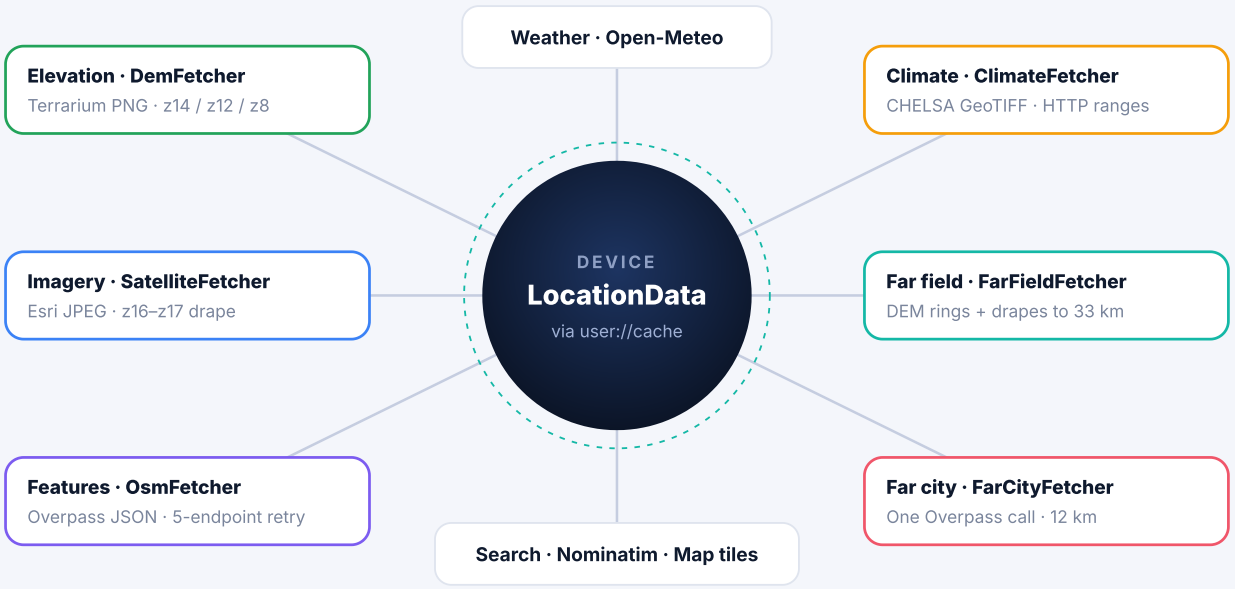


Figure 3. Fetch topology. Every fetcher writes a versioned cache file first and normalises into the `LocationData` contract; generation code never sees raw provider payloads.

NEED	SOURCE	FORMAT & ACCESS	LICENCE	FALLBACK
Elevation	AWS Terrain Tiles	PNG, h = R·256 + G + B/256 − 32768 m	Attribution (EU-DEM, USGS, SRTM…)	lower zoom → cache → procedural DEM
Imagery	Esri World Imagery	JPEG 256 px tiles, bounded pool	Esri ToU, noncommercial	lower zoom → cache → vertex colour
Buildings, roads, land	OSM Overpass	JSON `out geom`, identifying UA	ODbL	cache → absence-only dressing
Climate	CHELSA V2.1	GeoTIFF row reads, ~0.3–0.6 MB/place	CC BY 4.0	latitude rules (ClimateModel)
Weather	Open-Meteo	JSON current conditions	CC BY	omit weather dressing
Search / map	Nominatim · OSM tiles	JSON ≤1 req/s · PNG tiles	ODbL	gazetteer · blank map + pin
Textures	Poly Haven (build time)	13 PBR sets, 1k jpg in `assets/`	CC0	procedural textures

04 · PIPELINE

Generation pipeline

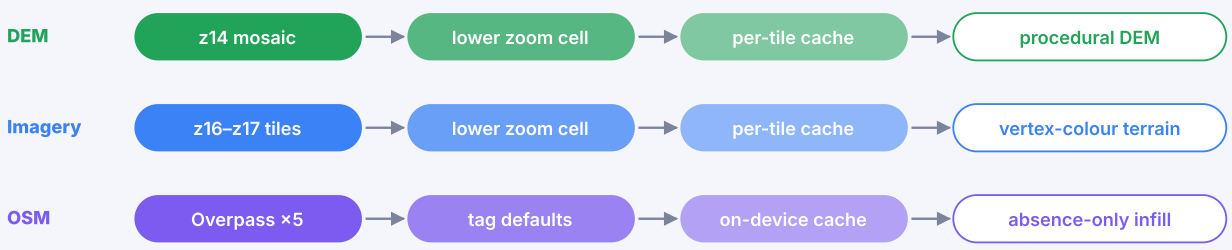
`GenerationPipeline` (`scripts/generation/pipeline.gd`) runs five frozen stages. All network fetches start together in `run()` and overlap; each stage awaits only what it needs, with a bounded wait so a slow source degrades a layer instead of blocking the view.



Figure 4. Overlapped fetch timeline (schematic). Stage rows are emitted as `stage_changed(i, label, "in_progress" | "done")`; `progress_changed` reports real items done/total, never a timer.

Fallback chains

Fallbacks run only on absence of real data, in a fixed order per layer. They log with `print` (an expected outcome offline), so a permanently broken endpoint is found by grepping for *fetch failed*.



After stage 3 the pipeline constructs `GroundTruth` once and `View360` builds the world on a background thread. A cancel bumps the generation id in `main.gd`, so a superseded world is freed rather than shown.

World model

The world-model contract (`docs/world-model-contract.md`) freezes three things: the `LocationData` fields, the pipeline stage order, and preset semantics. Changing any of them needs a contract task first.

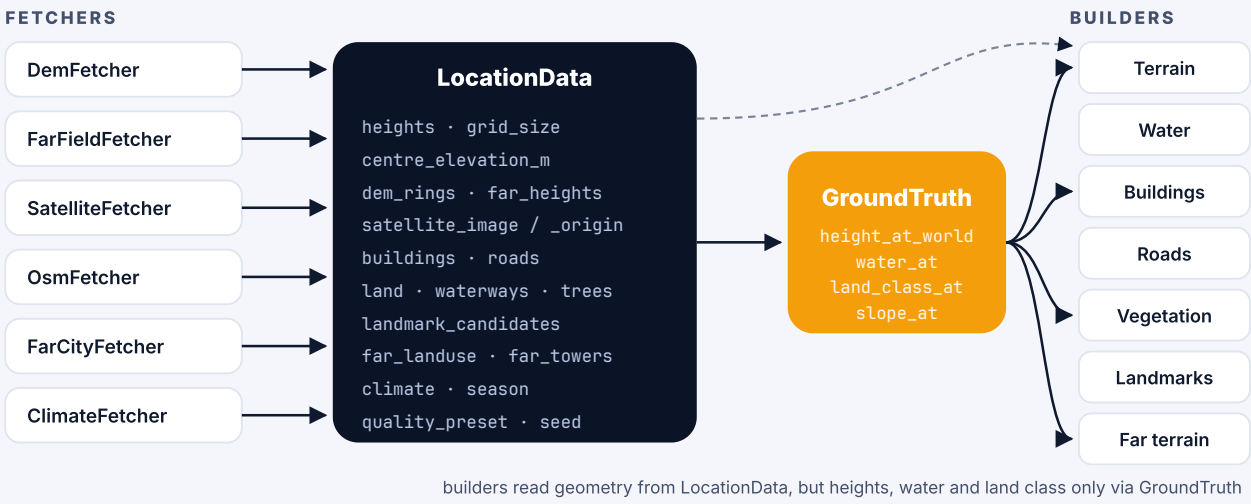
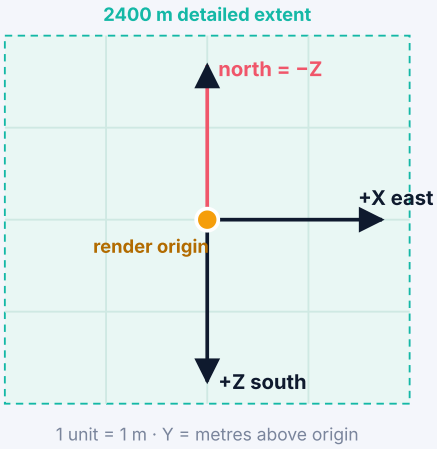


Figure 5. Data contract. Builders take `(data: LocationData, ground: GroundTruth)`, emit metre-space `Node3D` subtrees, and build materials only through `MaterialLibrary`.

Units and the render origin

`WorldScale` owns every conversion. One unit is one metre horizontally *and* vertically — the v1 220-unit world and adaptive vertical squash are gone. `+X` is east, `+Z` is south (north is `-Z`).

The detailed world is **2400 m** across (`DETAILED_EXTENT_M`). The render origin is set once per place at the viewpoint (lat, lon, `centre_elevation_m`), so the camera always works near the floating-point sweet spot.



GroundTruth

Built once per place from DEM posts, OSM polygons and waterways: a smoothed surface flattened only where OSM says so (building pads, roads) with water carved in. All samplers are pure bilinear reads — outside the extent they clamp to the nearest edge and never return NaN.

06 · ASSEMBLY

World assembly

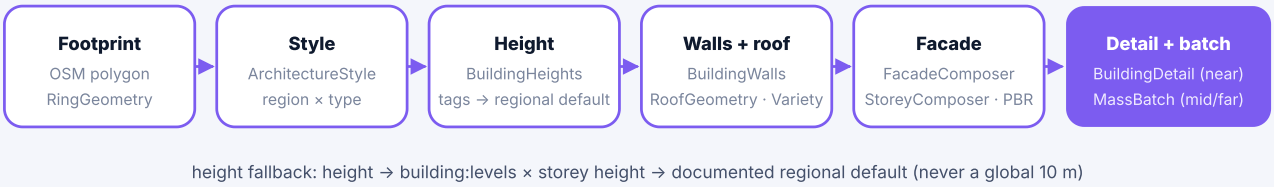
`TerrainBuilder.build_world(data, ground)` assembles the scene in dependency order. Later layers sample the surfaces earlier ones define, so trees never land on roads, buildings or water.



Figure 6. Build order. Each builder returns a contract subtree; `_adopt_flat` lifts its children into the root and records the layer they came from.

Building subsystem

Buildings are the largest subsystem (≈20 classes). Real OSM footprints are extruded first; procedural infill is **absence-only and style-matched** (`InfillPlanner`, `SettlementMap`) and never contradicts mapped data.



Terrain, water, roads

- **Terrain** — 96–256 mesh posts per side by preset over the 256-post `GroundTruth`; satellite drape and land-class tint in `GroundMaterial`'s shader.
- **Water** — triangulated OSM water, width-aware waterways, coastlines, shore aprons and stone quays where OSM implies built waterfront.
- **Roads** — attributed OSM roads as draped ribbon meshes; street furniture and barriers from `StreetProps`.

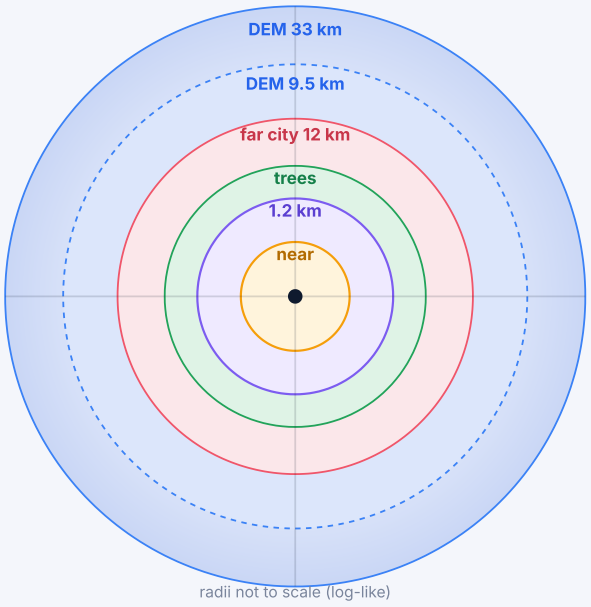
Vegetation & climate

- `ClimateModel` is the sole interpreter of CHELSA normals: snow line per season, treeline, biome and summer dryness.
- `TreeSpecies` picks a species mix from climate and land cover; `TreeMeshLibrary` builds procedural meshes, instanced near and impostors far.
- `VegetationMask` rasters keep trees and props off roads, buildings and water.

07 · RENDERING

LOD & rendering

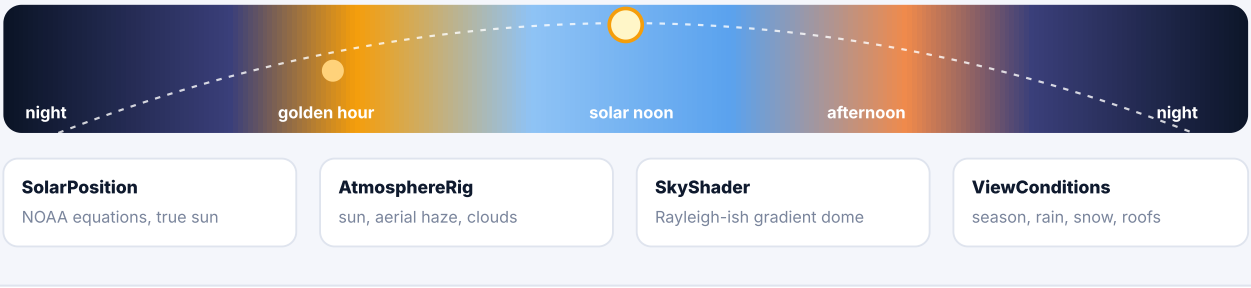
The 360 camera never translates, so level of detail is static: `LodRing` writes Godot visibility ranges once per build, with a 12% fade hysteresis. A preset switch only re-runs `apply()` — nothing is rebuilt, moved or recoloured.



RING	REACH	CONTENT
near	200–700 m	Full geometry with roofline detail (chimneys, dormers, rooftop plant)
mid	to 1.2 km	Building masses merged per material and 400 m cell; shared distant facades
trees	600–1700 m	Instanced species meshes, impostors past the cap
far city	4.5–12 km	Batched textured masses per octant × distance band, real towers ≥ 50 m
far DEM	to 33 km	~12.5 m/px to 4 km, ~25 m/px to 9.5 km, ~100 m/px to 33 km, with imagery drapes

Reach ranges span Fast → Ultra (`DETAIL_M`, `TREES_M`, `FAR_CITY_M` in `lod_ring.gd`). Draw calls scale with cells × materials, not with buildings.

Atmosphere, sun and weather



The view opens on the place's golden hour; the time slider and "now" button move the true solar position for the location's date. Live Open-Meteo weather drives cloud cover and precipitation, and `RoofSnow` and `ClimateModel` lay snow on roofs and ground from the season and CHELSA snow days. `RenderQuality` adds the photoreal layer per tier (SSAO, volumetrics, window recesses on Ultra).

08 · QUALITY

Quality presets & governor

One user choice matters: the render preset. It trades accuracy for time — loading time paid once, frame time paid every frame. `GroundTruth` resolution is preset-independent, so switching presets never moves objects.

	FAST	BALANCED	HIGH	ULTRA
Fidelity level	1	2	2	3
Terrain mesh posts / side	96	128	192	256
Satellite zoom	z16	z16	z17	z17
Max real buildings	600	1 200	2 500	6 000
Trees (urban / rural)	70 / 90	120 / 170	170 / 230	380 / 500
Render scale	65%	85%	100%	125%
Shadows (distance)	off	2.4 km	2.4 km	4 km
Cloud octaves	3	4	5	5
First build / revisit	**≤20 s / ≤3 s**	**≤45 s / ≤5 s**	**≤75 s / ≤5 s**	**1–2 min / ≤8 s**

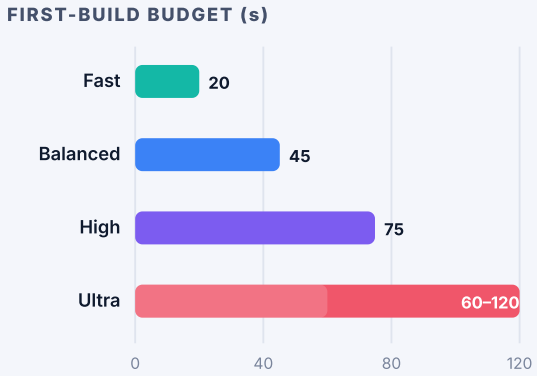


Figure 7. Broadband first-build budgets (plan §3.1). Cached revisits load the finished world instead.

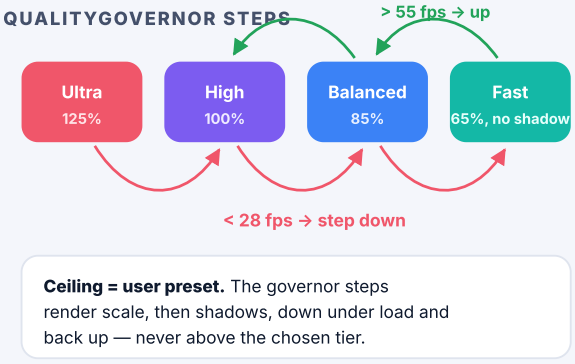


Figure 8. FPS-driven stepping. The HUD pill shows the tier plus the reduced render scale while stepping down.

09 · CACHE

Caching & offline

Two independent caches with separate budgets: the **data cache** holds fetched tiles and responses, the **world cache** holds finished worlds. A ~200 MB world can never evict the DEM, OSM or satellite tiles it was built from.

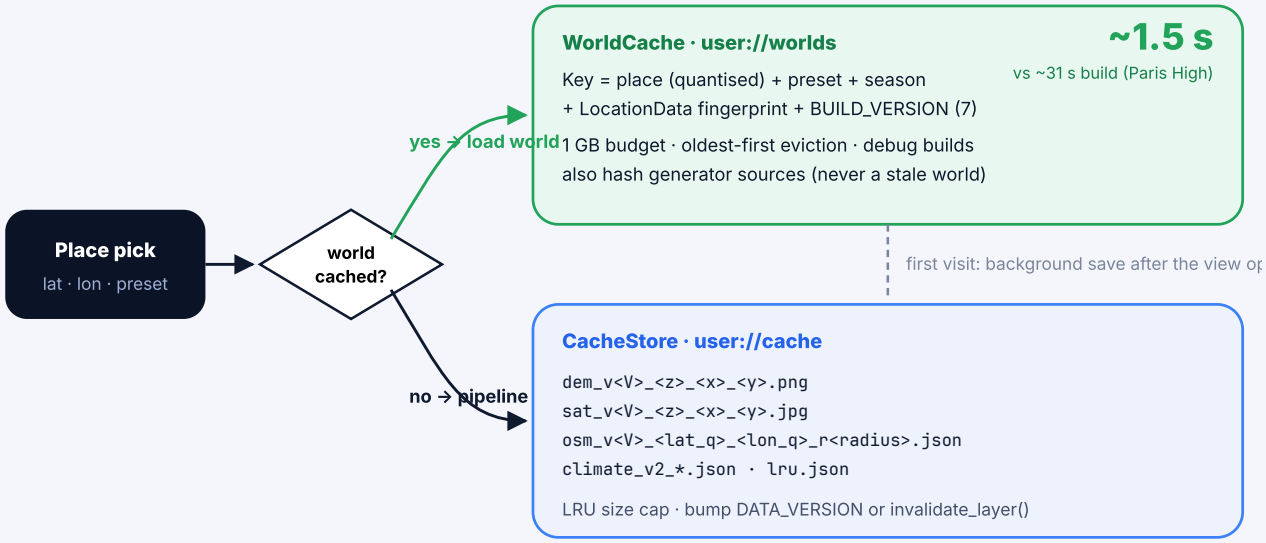


Figure 9. Revisit path. Headless test runs (dummy renderer) bypass the world cache entirely.

History

`CacheStore` is also the history store: a JSON file in `user://`. Generated terrain is reproducible from the stored coordinates and seed, so history entries double as cache keys and carry their v2 key set.

Versioning

Legacy v1 keys (`dem_`, `sat_`, `osm3_`) are ignored, never migrated; the LRU reaps them first under cap pressure. Area keys quantise coordinates to `round(deg × 1000)`.

Offline download

"Download for offline" in the picker runs `OfflineDownloader`, which warms every on-device cache for a place — DEM, imagery, OSM, far field and climate — with progress in the picker and a history entry on success.

> Offline gate: with networking disabled, a cached place loads in seconds and renders identically. An uncached place offline still renders, falling back to the deterministic procedural DEM.

Landmarks

Recognisable structures come from two paths. A **curated registry** of famous landmarks is modelled on the fly in true metres; any other notable OSM feature near the viewpoint is matched to an **archetype** (church, tower, dome, stadium…).



Figure 10. Curated registry (`LocationData`). Each entry has a trigger radius (2.5–13 km); inside it, the landmark is placed at its true bearing and distance.

LANDMARK	CLASS	HEIGHT
Eiffel Tower	`LandmarkEiffel`	330 m
Tokyo Tower	`LandmarkTokyo`	332.9 m
Elizabeth Tower	`LandmarkWestminster`	96 m
Colosseum	`LandmarkColosseum`	48 m
Sagrada Família	`LandmarkSagrada`	as built 2026
Giza · Liberty	`LandmarkMonuments`	true metres
Sydney Opera House	`LandmarkOpera`	true metres
Helsinki · Turku cathedrals	`LandmarkCathedrals`	true metres

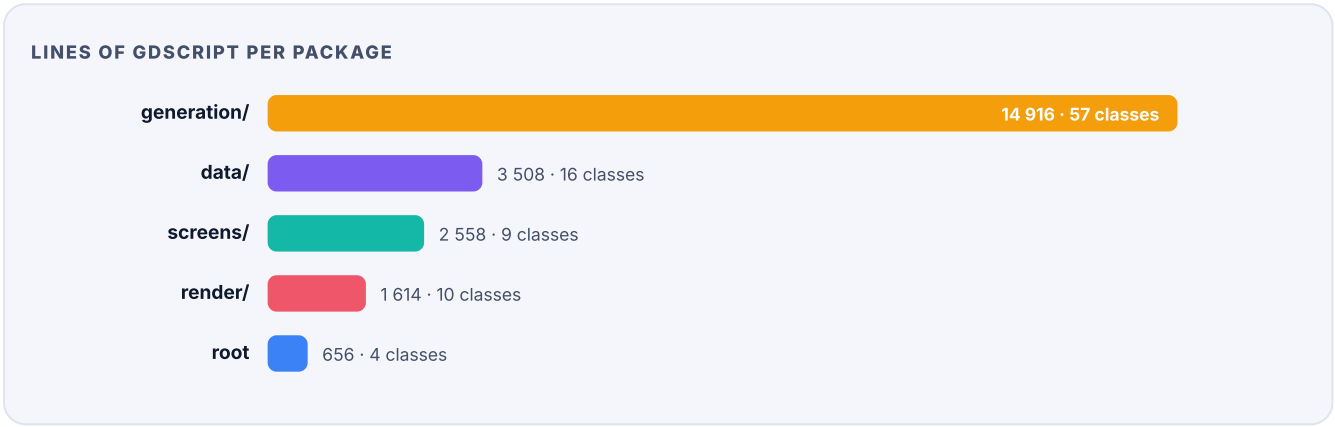
How a landmark is made

- `LandmarkMesh` accumulates merged geometry; `LandmarkMaterials` supplies calibrated photographic PBR materials.
- Models are generated on device — no downloads. `assets/eiffel.glb` is the Blender pipeline proof (`assets_src/blender/`).
- Up to five nearest OSM candidates per view become `LandmarkArchetypes`, within the haze limit.
- The registry landmark stays the hero; the spawn view frames it from a readable distance.

11 · CODE

Code organisation & testing

Every class is one GDScript file under `scripts/` and stays at or under **500 lines**; `tools/check_class_size.sh` fails the build on any violation. Large concerns split into focused helpers (for example `building_builder` → walls, heights, detail, roofs).



Package responsibilities

`data/`	Fetchers, parsers, contract, cache, history
`generation/`	Pipeline, ground surface, every builder, materials
`render/`	Camera, atmosphere, LOD, governor, presets, world cache
`screens/`	Picker, map tiles, loading, 360 view, history, settings
`scripts/*.gd`	Screen manager, theme, HUD style, responsive rules

Conventions

- Each class opens with a `##` doc comment stating its responsibility and the contract or plan task it implements.
- Static builders with `(data, ground)` signatures; heavy mesh work on a background thread via `MeshArrays`.

Test suite (31 headless scripts)

AREA	TESTS
Contracts & flow	smoke, flow, class_registry, build_guard, leak
Data parsing	osm, dem, tiles, search, weather, climate
World & scale	world_scale, terrain_scale, scenery, architecture, facade_relief, far_city, lod
Sky & light	sky, solar (vs ephemeris)
UI	responsive, zoom_search, map_pan, progressive_map, exit_tile, settings, loading_building
Landmarks	landmark
Windowed	screenshot, world_cache, fidelity_scorecard

Run any test with `godot --headless --script tests/<name>.gd`; windowed tests drop `--headless`. The fidelity scorecard (`docs/fidelity-scorecard.md`) checks golden locations — Paris, Interlaken, Banff, Big Sur, Turku.

12 · GALLERY

Gallery & attribution

Ultra-tier captures from the photoreal review (`visual-plan/reviews/photoreal-ultra-2026-10-30/`).



Paris, along the Seine — real OSM footprints, regional Haussmann facades, quays and water.



Interlaken valley — z14 DEM, far-field rings to the horizon, climate-driven snow line and vegetation.





Left: per-building roof variety. **Right:** window recesses on Ultra.

Attribution

© OpenStreetMap contributors (ODbL) · Elevation: AWS Terrain Tiles / Tilezen (EU-DEM / Copernicus, USGS 3DEP, SRTM and others) · Imagery: Esri, Vantor, Earthstar Geographics and the GIS User Community · Climate: CHELSA V2.1 (Karger et al., 2017, CC BY 4.0) · Weather: Open-Meteo (CC BY) · Search: Nominatim · Textures: Poly Haven (CC0) · Foliage textures: own procedural work (CC0). Code distributed under the MIT License.